



Programación II – COM 4

Trabajo Práctico Integrador

1er. Cuatrimestre 2026

SEGUNDA PARTE

Docentes

- Nores, José
- Gabrielli, Miguel A.

Integrantes

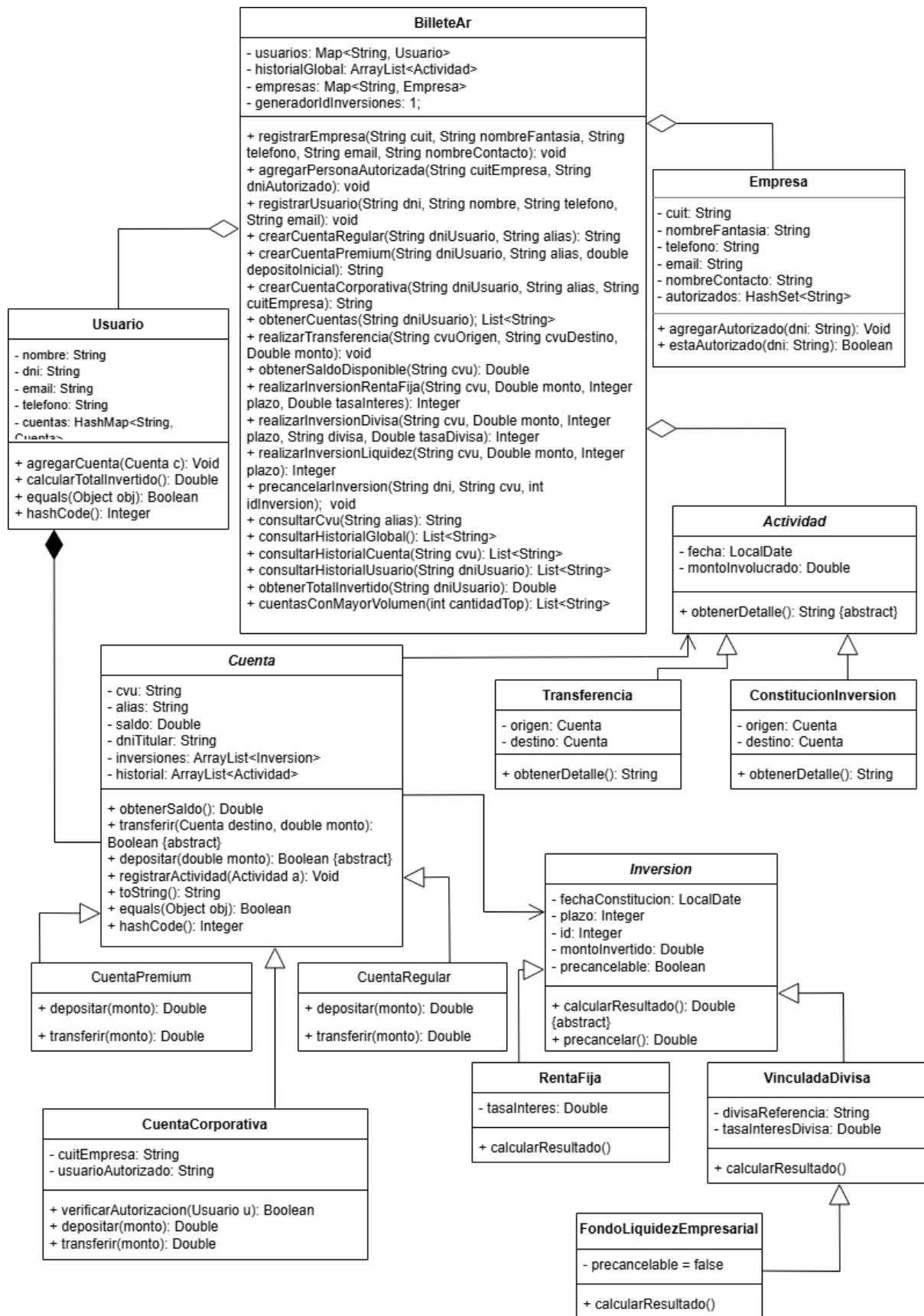
- Lupino, Lucas
 - Mendoza, Tomas
-

Introducción:

El sistema "billete.ar" fue creado como una plataforma fintech que permite a los usuarios gestionar múltiples cuentas financieras bajo una misma billetera virtual. La interfaz propuesta busca maximizar la flexibilidad operativa, permitiendo que cada tipo de cuenta (Regular, Premium, Corporativa) posea sus propias reglas y beneficios sin comprometer la integridad del sistema.

En primer lugar, separamos totalmente el funcionamiento interno de las reglas del negocio de la parte visual que el usuario maneja. Por otro lado, diseñamos el programa de manera que sea sencillo añadir nuevas funciones, como nuevos tipos de cuentas bancarias o diferentes opciones de inversión, sin tener que modificar lo que ya está funcionando. Finalmente, el sistema no pierde tiempo haciendo cálculos desde cero cada vez que alguien consulta sus ahorros, se mantiene un registro siempre actualizado del total invertido por cada persona.

Diseño:



Interfaz:

Gestión de Entidades y Empresas:

- **public void registrarEmpresa(String cuit, String nombreFantasia, String telefono, String email, String nombreContacto)** // Registra una nueva empresa en el sistema para permitir la apertura de cuentas corporativas.
- **public void agregarPersonaAutorizada(String cuitEmpresa, String dniAutorizado)** // Vincula a una persona (mediante su DNI) como autorizada para operar en nombre de una empresa.
- **public void registrarUsuario(String dni, String nombre, String telefono, String email)** // Registra a una persona nueva en la plataforma (ahora incluyendo el teléfono como dato obligatorio).

Gestión de Cuentas:

- **public String crearCuentaRegular(String dniUsuario, String alias)** // Abre una nueva cuenta regular y devuelve el CVU generado como String.
- **public String crearCuentaPremium(String dniUsuario, String alias, double depositoInicial)** // Abre una nueva cuenta premium validando el depósito inicial y devuelve su CVU.
- **public String crearCuentaCorporativa(String dniUsuario, String alias, String cuitEmpresa)** // Abre una nueva cuenta corporativa vinculada a una empresa y devuelve su CVU.
- **public List<String> obtenerCuentas(String dniUsuario)** // Devuelve un listado con el formato "[Tipo]: [Alias] ([CVU])"
- **public String consultarCvu(String alias)** // Dado un alias, consulta y devuelve el CVU asociado.

Operaciones Financieras e Inversiones:

- **public double obtenerSaldoDisponible(String cvu)** // Consulta la cantidad de dinero disponible en una cuenta específica.
- **public void realizarTransferencia(String cvuOrigen, String cvuDestino, double monto)** // Mueve dinero de una cuenta a otra, validando que existan y haya saldo suficiente.
- **public int realizarInversionRentaFija(String dni, String cvu, double monto, int plazoDias)** // Coloca fondos en renta fija y devuelve el ID único de la inversión.
- **public int realizarInversionDivisa(String dni, String cvu, double monto, int plazoDias, String divisa, double tasa)** // Coloca fondos en moneda extranjera especificando la divisa/tasa y devuelve su ID único.

- **public int realizarInversionLiquidez(String dni, String cvu, double monto, int plazoDias)** // Coloca fondos en un fondo de liquidez empresarial y devuelve su ID único.
- **public void precancelarInversion(String dni, String cvu, int idInversion)** // Permite cancelar anticipadamente una inversión activa utilizando su identificador único.

Consultas:

- **public List<String> consultarHistorialGlobal()** // Muestra todos los movimientos registrados en el sistema formateados como String.
- **public List<String> consultarHistorialCuenta(String cvu)** // Detalla las transacciones realizadas únicamente por una cuenta, devolviendo textos.
- **public List<String> consultarHistorialUsuario(String dniUsuario)** // Reúne el historial de todas las cuentas asociadas a una persona en formato de texto.
- **public double obtenerTotalInvertido(String dniUsuario)** // Devuelve la suma total de inversiones.
- **public List<String> cuentasConMayorVolumen(int cantidadTop)** // Identifica las "Top N" cuentas con mayor cantidad de movimientos realizados devolviendo sus detalles en texto.

IREP (Invariantes de Representación):

Sistema, Usuarios y Empresas

Clase BilleAr

- Para cada elemento en el mapa usuarios, la clave (String) utilizada debe ser exactamente igual al atributo dni del objeto Usuario que almacena como valor.
- Todo objeto Actividad presente en la lista historialGlobal debe existir simultáneamente en la lista historial de al menos una Cuenta registrada en el sistema.

Clase Usuario

- Los atributos nombre, dni y email no pueden ser nulos ni cadenas vacías.
- Para cada elemento en su mapa cuentas, la clave (String) debe ser exactamente igual al atributo cvu del objeto Cuenta que almacena como valor.

Clase Empresa

- Los atributos cuit, nombreFantasia, telefono, email y nombreContacto no pueden ser nulos ni cadenas vacías.
- La estructura de datos que guarda las autorizaciones debe estar obligatoriamente inicializada (no ser nula) al momento de crear la instancia de la empresa.
- Ningún elemento (String DNI) dentro de la colección de personas autorizadas puede ser nulo o vacío.

Jerarquía de Cuentas

Clase abstracta Cuenta

- cvu y alias no pueden ser nulos ni vacíos.
- saldo debe ser siempre ≥ 0 .
- Las colecciones inversiones e historial deben estar siempre inicializadas (no nulas) al crearse la cuenta.

Clase CuentaRegular

- Su saldo debe ser en todo momento $\leq 5.000.000$.

Clase CuentaPremium

- Al momento de ser instanciada (apertura), su saldo inicial debe ser ≥ 500.000 .

Clase CuentaCorporativa

- Los atributos cuitEmpresa y usuarioAutorizado no pueden ser nulos ni vacíos.

Jerarquía de Inversiones

Clase abstracta Inversion

- fechaConstitucion no puede ser nula, plazo debe ser estrictamente > 0 .
- Al momento de instanciarse, el montoInvertido debe ser estrictamente > 0 y $\leq$ al saldo disponible de la cuenta origen.

Clase RentaFija

- tasaInteres debe ser ≥ 0 .

Clase VinculadaDivisa

- divisaReferencia no puede ser vacía ni null y tasaInteresDivisa debe ser ≥ 0 .

Clase FondoLiquidezEmpresarial

- Su atributo montoInvertido debe ser en todo momento $\geq 20.000.000$.
- Su atributo precancelable debe ser estrictamente false y no puede modificarse.
- Un objeto de esta clase solo puede existir dentro de la colección inversiones de una instancia de tipo CuentaCorporativa.

Jerarquía de Actividades (Historial)

Clase abstracta Actividad

- fecha no puede ser nula y montoInvolucrado debe ser > 0 .

Clase Transferencia

- Los objetos referenciados en origen y destino no pueden ser nulos.
- El cvu del objeto origen debe ser estrictamente distinto ($\neq$) al cvu del objeto destino (una cuenta no se puede transferir a sí misma).

Clase ConstitucionInversion

- Los objetos origen e inversionRealizada no pueden ser nulos.
- El atributo montoInvolucrado heredado de la Actividad debe ser exactamente igual al atributo montoInvertido del objeto almacenado en inversionRealizada.

Explicación de la implementación:

- **StringBuilder:** Se aplicó principalmente en la redefinición del método toString() de los TADs principales (por ejemplo, en Billetera y Cuenta). Debido a que la generación del reporte del estado del sistema requería concatenar múltiples líneas de texto en un ciclo repetitivo, se optó por utilizar StringBuilder para optimizar el manejo de memoria y evitar la creación innecesaria de objetos String inmutables.
- **Iteradores y Foreach:** Se utilizaron bucles for-each para recorrer mapas y colecciones de manera limpia a lo largo de toda la fachada del sistema.

Asimismo, se incorporó el uso de la interfaz `Iterator<E>` de Java para recorrer colecciones puntuales como el historial de actividades, demostrando el control manual del flujo de la colección.

- **Herencia, Clases Abstractas y Polimorfismo:** Se definió la clase `Inversion` como una clase abstracta, encapsulando los atributos comunes (monto, fecha, plazo, `preCancelabilidad`). El comportamiento polimórfico se logró mediante la definición del método abstracto `calcularRetornoAlPrecancelar()`. Las clases hijas, como `RentaFija` y `VinculadaDivisa`, aplican sobreescritura (`@Override`) para proveer la lógica matemática específica que requiere cada tipo de instrumento financiero.
- **Interfaces y Sobrecarga:** Se cumplió con el uso de Interfaces mediante la implementación de `IBilletera`, garantizando un contrato claro para el código cliente. Además, se evidencia el uso de sobrecarga de métodos en la clase de `Utilidades` (propuesta por los profesores), permitiendo definir la fecha actual pasándole como argumento tanto un objeto `LocalDate` como un `String` representativo.

Análisis de complejidad:

```
// CLASE BilleAr
@Override
public void realizarTransferencia(String cvuOrigen, String cvuDestino, double monto) {
    Cuenta cuentaOrigen = buscarCuentaPorCvu(cvuOrigen);
    Cuenta cuentaDestino = buscarCuentaPorCvu(cvuDestino);

    cuentaOrigen.transferir(cuentaDestino, monto);

    Transferencia nuevaTransferencia = new Transferencia(cuentaOrigen, cuentaDestino,
monto);

    cuentaOrigen.registrarActividad(nuevaTransferencia);
    cuentaDestino.registrarActividad(nuevaTransferencia);

    this.historialGlobal.add(nuevaTransferencia);
}

private Cuenta buscarCuentaPorCvu(String cvu) {
```



```

    for (Usuario u : this.usuarios.values()) {
        for (Cuenta c : u.getCuentas().values()) {
            if (c.getCvu().equals(cvu)) {
                return c;
            }
        }
    }
    throw new IllegalArgumentException("Error: La cuenta con CVU " + cvu + " no existe.");
}

```

// CLASES Cuenta (Regular, Premium, Corporativa)

@Override

```

public Boolean transferir(Cuenta destino, double monto) {
    if (monto <= 0) { throw new IllegalArgumentException("El monto a transferir debe ser mayor a 0."); }
    if (this.obtenerSaldo() < monto) { throw new IllegalStateException("Error: Saldo insuficiente."); }

```

```

    this.setSaldo(this.obtenerSaldo() - monto);
    destino.depositar(monto);
    return true;
}

```

@Override

```

public Boolean depositar(double monto) {
    if (monto <= 0) { throw new IllegalArgumentException("El monto a depositar debe ser mayor a 0."); }
    // (Validación extra O(1) en caso de Cuenta Regular)
    this.setSaldo(this.obtenerSaldo() + monto);
    return true;
}

```

```

public void registrarActividad(Actividad a) {
    if (a != null) {
        this.historial.add(a);
    }
}

```

El proceso de transferencia se puede dividir en tres etapas principales: la búsqueda de las cuentas (origen y destino), la ejecución financiera de la transferencia y el registro histórico.

1. **Búsqueda de cuentas (buscarCuentaPorCvu):** El método utiliza dos ciclos for-each anidados. El bucle exterior itera sobre la colección de usuarios (sea U la cantidad de usuarios) y el interior itera sobre las cuentas de cada usuario (sea C la cantidad máxima de cuentas por usuario). En el peor de los casos (que la cuenta no exista o sea la última), se recorrerán todas las cuentas del sistema. Por lo tanto, la complejidad de buscar una cuenta es $O(U \times C)$, lo cual es equivalente a $O(N)$, siendo N el número total de cuentas registradas en el sistema.
2. **Transferir y Depositar:** Gracias al polimorfismo, se invoca el método correspondiente al tipo de cuenta. En todos los casos, el algoritmo realiza comparaciones lógicas (if), operaciones aritméticas simples (sumas y restas) y reasignaciones (setSaldo). Todas estas son operaciones elementales que se resuelven en tiempo constante, es decir, $O(1)$.
3. **RegistrarActividad y Add:** La creación del objeto Transferencia y el agregado al final de un ArrayList (this.historial.add) poseen un tiempo de ejecución amortizado constante, resultando en $O(1)$.

Justificación mediante algebra de Boole:

$$F(n) = F(\text{buscarOrigen}) + F(\text{buscarDestino}) + F(\text{transferir}) + F(\text{registroActividad})$$

Reemplazamos por sus complejidades individuales:

$$F(n) = O(n) + O(n) + O(1) + O(1)$$

Aplicando las propiedades del Álgebra de Órdenes:

$$F(n) = O(2n) + O(1)$$

$$F(n) = O(\max\{2n, 1\})$$

$$F(n) = O(n)$$